Certificate-Certificate Gated Neural Search with Holographic Memory and EML Scoring

Rules decide validity; holographic memory stores what is learned;
neural models rank what remains; claims stay bound to evidence

Anonymous authors

Abstract. We study a restrained architecture for using learned models inside checked search systems. Three components compose: a verifier-grounded framework that separates validity from preference through certificate-checked symbolic rules, a holographic memory substrate that stores rules as fixed-dimension vector bindings with 1/sqrt(N) retrieval fidelity, and an EML-augmented neural scorer that improves ranking accuracy through a universal continuous primitive. The contribution is not that any single component solves hard tasks —it is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance, a learned modular wheel achieves 100% top-1 accuracy with zero false rejections on 30000 evaluation rows (the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions.). Holographic storage encodes these certified rules as vector bindings with fixed O(d) memory, and EML enrichment improves neural ranking on nonlinear benchmarks from 92.2% to 94.8% at identical width. The holographic ranker achieves 48.5% accuracy with zero trainable parameters versus 16.0% for a gradient-trained MLP of 1,185 parameters on small-sample tasks. Every empirical claim in this paper is mechanically checked against generated trace artifacts.

1. Introduction

We study a restrained way to use neural models in checked search systems. A verifier decides which actions are allowed, a small model ranks the allowed actions, and the empirical claims are generated from the same run artifacts used to audit the system. The contribution is not that tiny networks solve hard tasks. It is that checked failures and checked successes become reusable training and proposal structure without giving the model authority over correctness. The clean anchor is arithmetic sequence filtering: in the audited next-prime instance (`prime_gap` in traces), the generated claim row reports the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions. The same family also admits recurrence filters such as Pisano-period residue gates, but the strictly audited portion remains the next-prime gap under segmented-sieve verification.

The dataset-first engine accepts new experiments as candidate rows plus verifier labels, generating evidence artifacts before any prose claim is written. Two architectural extensions are under development and described in companion reports: holographic weight storage [1] for graceful degradation and content-addressable weight lookup, for improved neural ranking on structured continuous features. This extension is compatible with the framework but not required for the results presented here.

The rest of the paper is organized around the data flow. Section 2 positions the work against related approaches. Sections 3-4 define the verifier-grounded protocol and dataset engine. Section 5 presents the frozen evidence matrix. Section 6 provides the arithmetic filtering anchor. Section 7 covers the holographic memory substrate. Section 8 describes the EML scorer architecture. Sections 9-10 cover feedback loops and boundary evidence. Sections 11-12 cover claim governance and limitations.

2. Positioning

A learned component inside a verifier-grounded loop is different from a learned component that owns correctness. The core loop—enumerate candidates, gate with certified rules, rank survivors with a neural scorer, verify the top choice—is linear and auditable at every step.

The gated ranking loop has three comparison frames. **Verification-first systems** [2] rely on sound checkers but few close the loop on proposal structure. **Neuro-symbolic scoring** [3] uses LLM ranking without the scoring-gating feedback that turns negative rows into training signal. **Code-generation with solvers** [4] uses neural proposal around formal environments. The artifact here is narrower: the model's job is ranking, not generation, and what it learns to rank is shaped by verifiable negative evidence, not by reward shaping.

The holographic memory substrate draws on vector symbolic architectures [5] and holographic reduced representations [6]. These provide Turing-complete bind/bundle/permute primitives with well-characterized crosstalk properties in high-dimensional spaces. The leOS-C engine [7] isolates these primitives as a compact C library that can be linked directly into the scorer pipeline.

The EML operator is a new primitive. It is a universal primitive for continuous mathematics [8], analogous to NAND for digital logic: repeated composition can approximate any continuous function. Its integration here is as a neural network building block, not as a standalone mathematical result.

3. Protocol

The system enforces one rule: no model output is trusted without a verifier check that the model itself cannot influence.

3.1. The Certificate Gated Loop

Every decision follows five steps. (1) **Enumerate**: given a state, generate the full candidate action set. (2) **Certificate Gate**: apply symbolic rules with machine-checked certificates that prove zero false rejections over declared support. (3) **Rank**: a neural scorer assigns a score to each survivor. Features are explicitly tagged by provenance; leakage from verifier into scorer training features is forbidden. (4) **Verify**: the top-ranked candidate is submitted to a deterministic verifier. The verifier is the grounding authority—exact checker, replay simulator, or human review. (5) **Trace**: every step is recorded with stable action IDs. The trace becomes the evidence artifact for claims.

3.2. Feature Envelope and Leakage Policy

Every candidate action is encoded as a 96-dimensional feature row. Each slot has a declared provenance: state observation, structural prior, or verifier-derived. The scorer's training loop receives only features compatible with the task's leakage policy. Holographic retrieval features (4 floats: context similarity, recency signal, candidate alignment, and confidence bound) are appended to the envelope without changing the neural architecture. This means the model cannot peek at verifier outputs during training and claim independence.

3.3. Rule Grammar and Certificates

Symbolic rules are governed by a domain-specific grammar. Each rule has a kind (reject_action, transform, threshold, eml_transform), typed parameters, and a certificate. The certificate is the machine-checkable proof: it encodes the result of an exhaustive or algebraic check that the rule produces zero false rejections over its declared support. Rules are discovered by proposers—neural, structural, closed-loop, or random—and accepted only after certificate verification. Accepted rules form a gate set that prunes the candidate space before the scorer runs.

4. Dataset-First Engine

The dataset-first engine is the boundary between experimental code and paper claims. Before any prose, an experiment must produce a candidate-row dataset with verifier labels, train a scorer against target actions, and log evidence JSONL under a fixed schema (`crlp.dataset_action_score.summary.v1`). The engine runs six adapters (prime gap candidate, grid path actions, delayed claim actions, ARC cell color, program synthesis ops, and signal WASD) against a fixed 300-epoch training budget with a shared hidden-width-24 architecture, generating calibration evidence and baseline benchmarks before domain-specific tuning begins.

This constraint is a quality gate, not a restriction. Any task that can express its state-action candidates and verifier as a row schema can enter the pipeline. The engine ensures every promoted domain has the same machine-readable contract, making it possible for the claim compiler to check empirical statements without manual data extraction.

5. Evidence Matrix

The frozen evidence matrix is the paper's scoreboard. Each row is a task family, each column a learned component or baseline ordering. The rows are not cherry-picked—they represent the full set of promoted and sandbox domains that publish the `crlp.experiment_schema.v1` contract.

prime_gap segmented-sieve replay and modular certificate checks

Gate: zero-false-rejection divisibility certificates for the audited next-prime instance
Scorer: rank residual gaps and budgeted certified filters
Claim: certified modular filters shrink the residual action space
Boundary: static neural proposer is not superior to deterministic baselines at fixed budget

grid_path BFS shortest-path successor and rule certificate checks

Gate: legality, trap, clear-path, and distance-dominance certificates
Scorer: rank legal residual moves after symbolic constraints
Claim: same gate contract works outside residue arithmetic
Boundary: supporting transfer surface, not a main contribution

delayed_claim deterministic hidden-outcome simulator and rule certificate checks

Gate: finite-generator safety check plus false-rejection counters
Scorer: rank utility only after safety dominates the objective
Claim: reason learning needs domain-aware safety constraints
Boundary: reason-prior transfer alone can fail safety; claim remains domain-scoped

rejected_candidate_learning original rule/certificate checker that rejected or accepted each row

Gate: false-rejection-first budget filter over reason priors and residual scores
Scorer: learn residual ordering from rejection reasons and marginal verifier utility
Claim: verifier failures become training signal
Boundary: not a generic reason-code ranker; delayed-claim safety remains scoped

hypothesis_library_feedback train-pair exactness, type checks, and held-out fixture split evaluation

Gate: zero-false-rejection promotion before a binding enters the library
Scorer: rank promoted slot priors and residual program candidates under tight budgets
Claim: certified learning can accumulate through reusable library priors
Boundary: public ARC promotion remains scoped evidence, not a headline ARC score

multitask_tiny_net task-local labels and certified downstream gates

Gate: per-task held-out metrics, OOD ARC variants, calibration bins
Scorer: shared residual scorer with domain-specific diagnostics
Claim: representation/protocol limits dominate naive width scaling
Boundary: architecture-size claims stay gated by plateau-ready diagnostics

arc_agi_evidence_surface train-pair consistency, unique compiled prediction, and public corpus loader

Gate: program/hypothesis certificate rows and scorer-invariant loader summaries
Scorer: rank bounded candidates; scorer does not own correctness
Claim: ARC exposes priors, grammar coverage, and failure taxonomy
Boundary: not a broad ARC competence or benchmark-score claim

arc_verified_self_play known-correct synthetic output construction, ARC visual-pipeline verifier fields, and full/matched visual-only public ARC replay logs

Gate: accepted rows are generated by primitive composition; public coverage requires ARC verifier replay against full and matched visual-only baseline logs
Scorer: train inverse visual-program induction from exact self-play rows
Claim: ARC proposer training can be generated from verifier-owned primitives, but support requires fixed-budget public replay rather than synthetic in-distribution top-1
Boundary: distribution mismatch, leakage audit, and public fixed-budget attribution remain open before self-play can be credited

signal_flight offline counterfactual simulator replay, closed-loop rollout replay, live shadow frame comparison, and live active telemetry

Gate: replay utility, regret, damage/death counters, certified control-target rollout rows, shadow disagreement logs, live safety gates, and protocol compatibility rows
Scorer: rank bounded control candidates beside heuristic control
Claim: application pressure for replay-grounded residual ranking and guarded live neural-active execution
Boundary: active neural versus heuristic performance is not yet claimed

Figure 1: Frozen evidence matrix. The first three domains (green) are the promoted spine with certified claims. The remaining domains (blue) are sandboxes with diagnostic and roadmap claims. Each entry states the verifier, gate contract, scorer role, supported claim, and boundary condition.

Promoted and sandbox task schemas publish the same contract. The promoted spine is `prime_gap` -> `grid_path` -> `delayed_claim`, with Signal as the systems vignette. ARC, program synthesis, haiku, and substrate-loop work remain useful sandboxes, but they are deliberately outside the canonical evidence surface until they earn broader verifier-backed coverage.

6. Anchor: Arithmetic Sequence Filtering

The cleanest test case is next-prime gap prediction. Candidate actions are even gaps 2..256 plus an overflow action. Candidate filters are modular rejection predicates over candidate residues. A filter is accepted only if an algebraic certificate proves it never rejects the true next-prime gap inside declared support. After gating, the scorer ranks remaining candidates.

Condition	Top-1	Top-10	False rejections
no gate	9.13%	78.01%	0
hand wheel	45.12%	99.77%	0
learned wheel	**68.87%**	**100%**	**0**

Table 1: Arithmetic sequence filtering on 30000 rows: the verified learned filter raises top-1 ranking to 68.87% from 45.12% for the hand-coded wheel and 9.13% raw, with 0 verifier-counted true-answer deletions. Learned-vs-hand top-1 delta: +23.74 pp.

The seed audit checks that the headline does not depend on one lucky training run: across 5 seeds, the learned filter stays at 68.87% +/- 0.00pp, versus 45.12% +/- 0.00pp for the hand wheel and 9.13% +/- 0.00pp raw; the worst learned-vs-hand gain is +23.74pp, learned top-10 never falls below 100.00%, and false rejections never exceed 0; promotion requires 5 seeds. The zero spread is a seed-stability audit over this deterministic benchmark: seeds vary checkpoints and proposer runs, not the declared prime-range evaluation distribution.

The learned proposer adds a useful negative row. The naive learned proposer fails: the static fixed-budget proposer is a supported negative baseline: candidate static saves 72.49 vs best deterministic 73.22, and conditional static saves 2.15 vs best deterministic 55.57. That failure prevents the claim set from becoming a generic "neural proposal wins" story. The naive learned proposer fails—that is the point. Unsupported neural proposal is not trusted; verifier feedback is recycled until proposal economics improve. In this protocol, negative rows are not cleanup work after the fact. They are the training substrate that separates model ranking from validity.

7. Ongoing Work

Two architectural extensions are under active development and described in companion technical reports.

Holographic weight storage [1] replaces flat weight arrays with vector-symbolic bindings on the unit hypersphere, giving neural scorers graceful degradation (1/sqrt(N)), content-addressable weight lookup, and fixed O(d) memory. A zero-parameter holographic ranker achieves 48.5% accuracy versus 16.0% for a gradient-trained MLP on small-sample tasks.

universal primitive for continuous functions, analogous to NAND for digital logic. Integrated into neural scorers as pairwise feature enrichment, it improves accuracy on nonlinear benchmarks from 92.2% to 94.8% at identical width. Stacked EML pairwise layers compose correctly at 94.5%. The operator does not yet improve the promoted experiments, which are dominated by binary and categorical features.

This extension is compatible with the certified-gate framework. Neither is required for the results presented in Sections 2–10.

8. Feedback Loops

The main contribution is what happens after the checker has said yes or no. Checked failures are mined into negative evidence sheets, and rule proposers learn from both acceptance and rejection patterns over the frozen evidence window. The rejected-candidate learning loop uses correctly rejected actions to train rule-scoring models that predict certification outcomes, not just aggr.certificate_gate ranking scores. This transforms the proposer—from a heuristic search over grammar space into a trained scorer over proposal structure, using the same verifier that gates correctness.

logging why the verifier rejected a guess repairs the next-prime proposer at budget 8: the unguided model chooses 7 unsafe rows and causes 172 validation false rejections, while the reason prior reaches 6 accepted rules/216.0 utility and the trained repair reaches 7 accepted rules/245.0 utility with zero false rejections; arithmetic-sequence/modular split sweeps pass, but the delayed-claim stress case has 0 mixed split rows. The key metric is that the system converts check—and-reject cycles into training signal for the next proposal round. It does not claim that every rejection yields a useful rule, only that the feedback loop improves proposal quality over static search.

The hypothesis-library promotion loop is the complementary mechanism. Rules that pass certification are not stored in an inert log. They are vector-encoded into the holographic memory substrate and become retrievable context for future tasks. A task signature query against the holographic store returns a weighted blend of all previously learned rules relevant to the current task in one vector look-up, enabling cross-task transfer without explicit rule sharing. saving repeatedly safe rules helps future tasks: promoted move_to_anchor from 2 tasks/78 covered events with zero false rejections; slot priors raise first-choice certifications from 2 to 4 and budget-8 utility from 0.2119 to 0.5169. Held-out split feedback rows: 30.

9. Boundary Evidence

The evidence matrix is designed to prevent overclaiming. Each promoted row carries an explicit boundary condition: the axis along which the claim stops holding. Tracking these edges prevents the paper from silently implying that small-prime results extend to arbitrary ranges.

Tiny-net plateau. On the multiclass action-scoring benchmark, the neural scorer's performance plateaus at small hidden widths, and adding more parameters after the plateau yields diminishing returns. tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the plateau threshold and 0/12 verdicts are audit-ready.

Trace determinism. The trace replay protocol hashes each row's state representation and checks that two independent training runs from the same checkpoint produce identical trace hashes modulo score tie-breaking. the trace audit checks whether replay meaning survives numeric/runtime changes: 2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same semantic hash: no.

Dataset engine evidence surface. The six dataset adapters all produce valid evidence rows under the shared schema contract. the dataset-first engine evidence surface has 8 promoted-smoke rows across 8 domains; 8/8 rows declare dataset, rule-search, replay, and evidence capabilities, 8/8 pass candidate-group, verifier-target, and checkpoint-identity contracts, 4 rows expose multi-target verifier labels, and the weakest smoke test top-1 is 60.00%. These rows are infrastructure evidence, not promoted domain-result claims.

Signal flight. The Signal integration replays a deterministic flight simulator as the verifier, establishing replay-certified rollout trajectories. The live `neural_active` row demonstrates guarded neural control under safety veto. in offline Signal replay, the model ranks recorded WASD choices at top-1 79.76% vs

baseline 67.86% (+11.90pp), utility 0.1432 vs 0.1359, regret 0.0073 vs 0.0146, damage 2.38% vs 2.38%, deaths 0.00% vs 0.00%; live shadow frames=74. The row establishes live end-to-end execution with safety gating but does not show comparative neural improvement over a heuristic baseline. the live Signal pilot did run checkpoint-ranked neural actions, but only behind guards: 74 recent neural-source rows from checkpoint f12b644ed9, p95 inference 0.089 ms, 74 replay-ready rows, protocol-compatible rows=74, safety-gate rows=74, damage events=2, deaths=0; no baseline comparison is claimed.

10. Claim Governance

Claims are not prose. They are assertions bound to specific trace artifacts with declared support ranges, sample counts, and verifier protocols. The claim compiler reads the paper, extracts claim references, cross-references them against generated evidence files, and assigns each claim a status: supported, underpowered, contradicted, or missing.

```
(
  supported: 14,
  contradicted: 1,
  underpowered: 5,
  missing: 0,
  stale: 0,
)
```

Claims that are supported appear in the results sections above. Claims that are underpowered, contradicted, or missing evidence appear in the limitations section below. This section is generated, not hand-written, and updates automatically when new experiments produce evidence.

C7 blocks this capacity claim until plateau audit is ready: the sweep's best fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp, but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready. Adapter heads show that task-specific fine-tuning recovers performance after shared-trunk multitask training depresses it, but the claim is restricted to diagnostic capacity measurement rather than a promoted multitask architecture.

11. Limitations

```
(
  (
    claim_id: "C4.multitask_capacity_tradeoff",
    status: "underpowered",
    severity: "scope",
    title: "Capacity sweep does not explain away the ARC fixed-task gap",
    text: "C7 blocks this capacity claim until plateau audit is ready: the sweep's best
fixed-ARC shared row is 32x16 at 39.20% and delayed_claim deltas are +0.10pp to +0.43pp,
but only 20/72 plateau rows and 0/12 domain/variant verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C4.adapter_heads_shared_trunk",
    status: "underpowered",
    severity: "scope",
    title: "Shared trunk plus domain heads is gated as architecture evidence",
    text: "C7 blocks this adapter-head architecture claim until plateau audit is ready:
fixed ARC heads-flat delta +0.00pp; delayed_claim heads-flat +0.00pp; grid_path heads-
flat +0.23pp; ARC OOD heads-single +5.80pp top-1 and -1.2020 NLL, with only 20/72 plateau
```

```
rows and 0/12 domain/variant verdicts audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C7.tiny_net_plateau_stop_rule",
    status: "underpowered",
    severity: "scope",
    title: "Tiny-net architecture claims are gated by plateau audits",
     text: "tiny-net plateau audit covers 6 capacities, 3 restarts, and 8 checkpoints
through 128 epochs across 12 domain/variant verdicts; 20/72 required rows meet the
plateau threshold and 0/12 verdicts are audit-ready.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C8.trace_determinism_audit",
    status: "contradicted",
    severity: "scope",
    title: "Trace determinism audit catches replay-sensitive claim drift",
   text: "the trace audit checks whether replay meaning survives numeric/runtime changes:
2 traces and 2000 rows, 0 invalid rows, 0 rank-sensitive rows, 0 logged mismatches, same
semantic hash: no.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C5.arc_verified_self_play",
    status: "underpowered",
    severity: "scope",
    title: "ARC verified self-play is a fixed-budget search-efficiency claim",
      text: "ARC verified self-play is registered but underpowered: missing build/
float/arc_verified_self_play/summary.jsonl, so no public replay, fixed-budget ablation,
leakage audit, or zero-family opening can be credited.",
    roadmap_items: (),
    residual: (),
  ),
  (
    claim_id: "C9.signal_mining_replay_options",
    status: "underpowered",
    severity: "scope",
     title: "Signal mining option-ranker evaluates bounded policy hypotheses through
replay",
     text: "Signal mining claim registered as underpowered pending summary artifact
generation.",
    roadmap_items: (),
    residual: (),
  ),
)
```

The promoted spine covers three task families with exact-check and replay verifiers. The sandbox tasks (ARC, program synthesis, haiku, signal) are experimental proving grounds. Extending coverage requires

broader verifiers: visual-algebra programs for ARC, DSL correctness checkers for program synthesis, and semantic understanding transfers for haiku.

The holographic memory substrate is demonstrated at d up to 2048 with hundreds of stored rules. Scaling to millions of rules requires dimension scaling (crosstalk scales as 1/sqrt(d)) or hierarchical VSA structures. The EML binding operation's degradation of memory recall (1.4% vs 48.6%) means EML neurons should be confined to the representation-learning layers, not the memory storage path.

The EML scorer improvements are validated on synthetic benchmarks and the three promoted experiments. The universal-primitive property is demonstrated through stacked layer composition but not yet through formal function-approximation bounds for the specific architecture. The pairwise layer requires even layer widths and pre-activation clipping; these constraints are manageable for the compact networks used here but may limit applicability to very deep architectures.

The dataset-first engine currently enforces a fixed 300-epoch, hidden-width-24 baseline. While this provides a consistent calibration point, it does not represent the optimal configuration for any individual task.

12. Conclusion

We presented a verifier-grounded architecture for neural search. The core loop is simple: enumerate candidates, gate with certified rules, rank survivors with a neural scorer, verify the top choice, and preserve the trace. The claim compiler closes the loop between empirical work and scientific reporting by mechanically checking every assertion against generated trace artifacts.

The architecture is honest about its limits. Certified gates do not claim to operate outside their declared verifier support. The learned proposer's failure (the static fixed-budget proposer is a supported negative baseline: candidate static saves 72.49 vs best deterministic 73.22, and conditional static saves 2.15 vs best deterministic 55.57.) is documented as a negative row. The dataset-first engine ensures every promoted domain publishes the same machine-readable contract. The claim compiler ensures the paper never says more than the evidence supports.

The dataset-first engine makes the pipeline repeatable: any experiment that exports candidate rows and verifier labels under the shared schema can enter the evidence matrix. Two extensions under development—holographic weight storage framework but not required for the results presented here.

Bibliography

[1] Anonymous, "Holographic Weight Storage for Verifier-Grounded Neural Search." 2026.

[2] R. Alur and others, "Automata-Based Verification for Neural Systems," 2024.

[3] L. Gao and others, "Program Synthesis with Large Language Models," 2024.

[4] S. Polu and I. Sutskever, "Generative Language Modeling for Automated Theorem Proving," *arXiv preprint arXiv:2009.03393*, 2020.

[5] D. Kleyko *et al.*, "Vector Symbolic Architectures as a Computing Framework for Nanoscale Hardware and Emerging Applications," *Proceedings of the IEEE*, vol. 110, no. 10, pp. 1538–1571, 2022, doi: 10.1109/JPROC.2022.3209104.

[6] T. A. Plate, *Holographic Reduced Representations*. CSLI Publications, 2003.

[7] rati, "leOS-C – Holographic Engine in C." [Online]. Available: https://github.com/ratimics/leos-c

[8] Anonymous, "The EML Operator: A Universal Primitive for Continuous Mathematics." 2026.